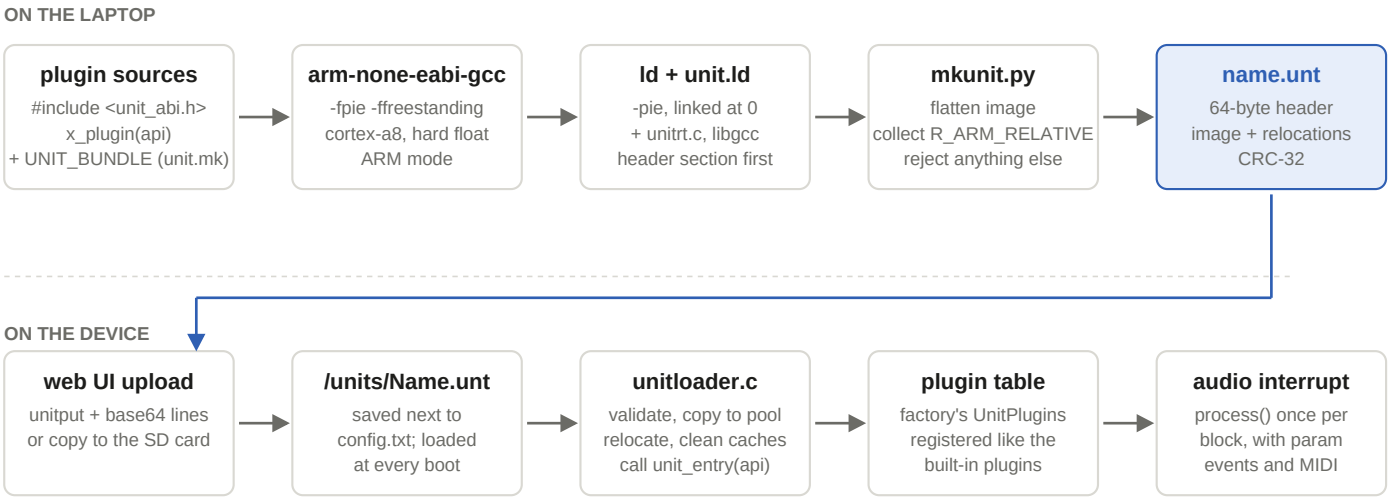
Metalhead unit ABI

Version 2 · loadable units for the metalhead firmware · 30 October 2026

A **unit** is a file of audio/MIDI plugins built on the laptop, separately from the firmware, and loaded by the running device. It is uploaded from the web UI (or copied to the SD card), stored as `/units/<Name>.unt` next to `config.txt`, loaded at every boot, and its plugins are then used in `[units]` like the built-in ones. Plugins and parameters follow CLAP: a factory, a plugin descriptor, parameter infos with plain value ranges, and parameter events. This document summarises the binary interface: what a unit file contains, how it is loaded, which functions run where, and the exact layout of every structure. The source of truth is `ccs/units/include/unit_abi.h`; offsets and sizes below come from the ARM compiler (BeagleBone Black; on the Raspberry Pi 5 pointers are 8 bytes).

ABI VERSION **2** exact match required	TARGET **Cortex-A8, ARM mode** AAPCS, hard float, NEON	CODE **Position independent** -fpie, freestanding, libgcc only
RELOCATIONS **R_ARM_RELATIVE only** base address added per word	HOST SERVICES **5 functions** log, allocate, 3 file reads	PLUGINS **Bundles, CLAP-style** 60-byte descriptor + vtable

From source to sound



There is no dynamic linker. A unit cannot call firmware or C library functions by name: everything it needs from the host comes through two tables of function pointers, **UnitHostApi** (passed once to unit_entry) and **UnitProcessHost** (passed to every process call). mkunit.py fails the build if the unit references anything outside its own image, so a missing symbol never reaches the device.

Lifecycle and calling contexts

The loader validates the file, copies the image into a 2 MB code pool in DDR, zeroes its bss, adds the load address to every relocated word, cleans the D-cache, invalidates the I-cache and branch predictor, calls unit_entry and registers every plugin of the returned factory, all or none. Units are never unloaded.

Call	When	Runs in	Allowed
`unit_entry(api), x_plugin(api)`	once, at boot (before config.txt) or right after an upload	main loop	everything: log, allocate, file access, slow set-up work (e.g. loading samples)
`createInstance()`	for each [units] line when config.txt is loaded	main loop	log, allocate, file access; return NULL when out of instances
`getNum*, getParamCount, getParamInfo, paramValueToText, paramTextToValue`	while loading the unit, resolving config.txt, and for the web UI	main loop	must be pure and return the same answers every time
`getParamValue(instance, ...)`	for the web UI (paramget)	main loop, interruptible by process()	read a value process() keeps up to date; don't compute
`process(...)`	once per audio block (one USB frame of samples)	**audio interrupt**	log and sendMIDIEvent only. No file access (refused), no allocate, no blocking or waiting

process() signature

```
void process(void * instance, UnitProcessHost * host,
             size_t nrInChannels,  float ** inSamples,    // unconnected inputs contain silence
             size_t nrOutChannels, float ** outSamples,   // outputs start zeroed; add into them
             size_t frameCount,                           // samples per channel in this block
             const UnitParamEvent * paramEvents, size_t paramEventCount);  // apply these first
```

Parameter values are plain. A parameter declares minValue..maxValue in its own unit (40..240 BPM, 0..50 ms) and events carry values in that range. MIDI controllers and [presets] still use 0..127: the firmware maps 0 to minValue and 127 to maxValue and rounds stepped parameters. Trigger parameters (IS_TRIGGER) are actions, such as a button or a drum hit with the velocity as value, and have no value for getParamValue. Unlike CLAP, parameters belong to the plugin, not the instance. At most 16 MIDI events are delivered per block for all units together. Floating point and NEON are fine in process(): the IRQ handler saves the full VFP/NEON state.

Structures

All structures are little endian with natural ARM EABI alignment unless noted. The firmware static-asserts that the size and every field offset match its own plugin structures (`unitloader.c`), so a unit's UnitPlugin goes straight into the plugin table.

UnitFactory 12 bytes (BBB)

Returned by unit_entry (clap_plugin_factory); UNIT_BUNDLE defines it. Must point into the unit's own image, as must everything below (the loader checks).

Offset	Size	Field	Type	Meaning
0	4	`size`	`uint32_t`	sizeof(UnitFactory) the unit was built with
4	4	`getPluginCount`	`uint32_t (*)(void)`	plugins in this file; fixed after unit_entry
8	4	`getPlugin`	`const UnitPlugin *(*)(uint32_t)`	plugin number index

UnitPlugin 60 bytes (BBB)

One per plugin in the file: the descriptor (clap_plugin_descriptor) and the functions, including the params extension.

Offset	Size	Field	Type	Meaning
0	4	`size`	`uint32_t`	sizeof(UnitPlugin) the unit was built with; later functions only called if present
4	4	`name`	`const char *`	type name used in [units], [A-Za-z0-9_]{1,31}
8	4	`vendor`	`const char *`	may be NULL
12	4	`version`	`const char *`	may be NULL
16	4	`description`	`const char *`	one line, may be NULL
20	4	`createInstance`	`void *(*)(void)`	one per [units] line; NULL if none left
24	4	`releaseInstance`	`void (*)(void *)`	a config reload dropped the unit; may be NULL (leaks)
28	4	`getNumInputs`	`uint32_t (*)(void)`	audio inputs unit:1..N
32	4	`getNumOutputs`	`uint32_t (*)(void)`	audio outputs unit:1..N
36	4	`getParamCount`	`uint32_t (*)(void)`	parameters (CLAP params.count)
40	4	`getParamInfo`	`int (*)(uint32_t, UnitParamInfo *)`	1 if the index is valid (params.get_info)
44	4	`getParamValue`	`int (*)(void *, uint32_t id, double *)`	an instance's current value; main loop (params.get_value)
48	4	`paramValueToText`	`int (*)(id, double, char *, uint32_t)`	optional: "12.5 ms"
52	4	`paramTextToValue`	`int (*)(id, const char *, double *)`	optional: parse what the user typed
56	4	`process`	`void (*)(...)`	once per audio block, in the audio interrupt (see above)

UnitParamInfo 96 bytes (BBB)

Filled by getParamInfo (clap_param_info without the cookie). The firmware checks every parameter when the plugin is loaded.

Offset	Size	Field	Type	Meaning
0	4	`id`	`uint32_t`	stable id, anything but UNIT_INVALID_ID; arrives in UnitParamEvent
4	4	`flags`	`uint32_t`	UNIT_PARAM_IS_*: CLAP's values, plus IS_TRIGGER (1 << 24)
8	32	`name`	`char[32]`	used in config.txt as unit:name, [A-Za-z0-9_.-]+
40	32	`module`	`char[32]`	group for user interfaces ("tank/damping"), or ""
72	8	`minValue`	`double`	MIDI 0 and preset value 0 map here
80	8	`maxValue`	`double`	MIDI 127 maps here
88	8	`defaultValue`	`double`	what a new instance starts with

UnitParamEvent 16 bytes (BBB)

A parameter change for process() (clap_event_param_value), from [midimap], [presets] or the web UI (paramset).

Offset	Size	Field	Type	Meaning
0	4	`time`	`uint32_t`	sample offset in the block; 0 for now
4	4	`id`	`uint32_t`	UnitParamInfo.id
8	8	`value`	`double`	plain value, minValue..maxValue (rounded if stepped)

UnitProcessHost 72 bytes (BBB)

Passed to every process() call; only valid during that call.

Offset	Size	Field	Type	Meaning
0	4	`play`	`void (*)(void)`	start the transport
4	4	`stop`	`void (*)(void)`	stop the transport
8	4	`requestTempo`	`void (*)(float bpm)`	change the tempo
12	4	`sendMIDIEvent`	`void (*)(hostData, UnitMidiPacket)`	queue MIDI on the unit's output ([midiroutes]); max 16 per block, all units together
16	40	`timeInfo`	`UnitTimeInfo`	timing for this block
56	4	`hostData`	`struct UnitHostData *`	opaque; pass to sendMIDIEvent
60	4	`midiIn`	`const UnitMidiPacket *`	MIDI routed to the unit by [midiroutes] since the last block; check UNIT_PROCESS_HOST_HAS
64	4	`midiInCount`	`size_t`	number of packets in midiIn

UnitTimeInfo 40 bytes (BBB)

Embedded in UnitProcessHost at offset 16.

Offset	Size	Field	Type	Meaning
0	8	`microSeconds`	`uint64_t`	device clock at the start of the block
8	4	`sampleRate`	`float`	Hz (48000)
12	4	`isPlaying`	`int`	transport running (Ableton Link / MIDI clock timeline)
16	8	`barStartPos`	`double`	start of the current bar, in quarter notes
24	8	`ppqPos`	`double`	position in quarter notes
32	4	`tempo`	`float`	BPM
36	4	`timeSigNumerator`	`int`	quantum (beats per bar)

UnitMidiPacket 4 bytes (BBB)

A USB-MIDI event packet. SysEx is sent as CIN 0x4 packets followed by a 0x5-0x7 end packet.

Offset	Size	Field	Type	Meaning
0	1	`header`	`uint8_t`	(cable << 4) | code index number; cable 0 = default jack
1	1	`midi0`	`uint8_t`	status byte (e.g. 0x90 note on, ch 1)
2	1	`midi1`	`uint8_t`	data 1
3	1	`midi2`	`uint8_t`	data 2

UnitHostApi 32 bytes (BBB)

Passed to unit_entry; stays valid and may be kept. Services added later are appended: check `UNIT_HOST_HAS(api, field)` before calling them, so the unit still loads on older firmware.

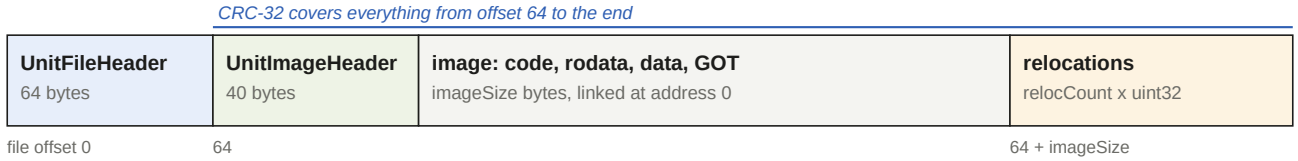
Offset	Size	Field	Type	Meaning
0	4	`abiVersion`	`uint32_t`	UNIT_ABI_VERSION of the firmware
4	4	`size`	`uint32_t`	sizeof(UnitHostApi) in the firmware
8	4	`log`	`void (*)(const char *fmt, ...)`	queued console log; safe from process()
12	4	`allocate`	`void *(*)(size_t)`	memory for the unit's life, 8-byte aligned, not zeroed; NULL when the 64 MB heap is full
16	4	`fileSize`	`int32_t (*)(const char *path)`	size of a file, < 0 if missing or a directory
20	4	`readFile`	`int32_t (*)(path, offset, buf, len)`	bytes read, < 0 on error
24	4	`readDirectory`	`int32_t (*)(path, index, entry *)`	1 = entry filled, 0 = no more, < 0 = error
28	4	`processHostSize`	`uint32_t`	sizeof(UnitProcessHost) in the firmware (see UNIT_PROCESS_HOST_HAS)

UnitDirectoryEntry 72 bytes (BBB)

Filled by readDirectory. Entries are returned in on-disk order; sort if order matters.

Offset	Size	Field	Type	Meaning
0	64	`name`	`char[64]`	file or folder name (longer names are skipped)
64	4	`size`	`uint32_t`	file size in bytes
68	1	`isDirectory`	`uint8_t`	non-zero for folders

The .unt file



The image is linked at address 0 and starts with the UnitImageHeader that UNIT_BUNDLE places in the .unit_header section. After loading at address *base*, the loader adds *base* to the pointer-sized word at each relocation offset. The firmware allocates imageSize + bssSize bytes, 64-byte aligned on the BBB and 4 KiB aligned on the Pi.

UnitFileHeader 64 bytes (BBB)

Written by mkunit.py.

Offset	Size	Field	Type	Meaning
0	4	`magic`	`uint32_t`	"UNT1" (0x31544E55)
4	2	`abiVersion`	`uint16_t`	must equal the firmware's UNIT_ABI_VERSION
6	2	`headerSize`	`uint16_t`	64
8	4	`imageSize`	`uint32_t`	bytes of image that follow, a multiple of 4
12	4	`bssSize`	`uint32_t`	zeroed bytes the loader adds after the image
16	4	`relocCount`	`uint32_t`	uint32 offsets after the image
20	4	`entryOffset`	`uint32_t`	offset of unit_entry in the image
24	4	`crc32`	`uint32_t`	IEEE CRC-32 (zlib.crc32) of everything after this header
28	4	`machine`	`uint32_t`	0 = 32-bit ARM (BeagleBone Black), 1 = 64-bit ARM (Raspberry Pi 5)
32	32	`name`	`char[32]`	unit file name (/units/<name>.unt), NUL-terminated, [A-Za-z0-9_]{1,31}

UnitImageHeader 40 bytes (BBB)

First bytes of the image; checked against the file header.

Offset	Size	Field	Type	Meaning
0	4	`magic`	`uint32_t`	"UNIT" (0x54494E55)
4	4	`abiVersion`	`uint32_t`	UNIT_ABI_VERSION the unit was built with
8	32	`name`	`char[32]`	unit file name; written by UNIT_BUNDLE("Name", ...)

What the loader checks

1	magic "UNT1", headerSize 64 and an exact ABI version match
2	sizes consistent with the file length; image a multiple of 4 bytes
3	CRC-32 over image and relocations
4	entry point and every relocation inside the image, 4-byte aligned
5	image header magic, ABI version and name match the file header
6	after unit_entry: the factory, every plugin, its strings and function pointers lie inside the unit's image
7	plugin size at least sizeof(UnitPlugin); valid and unique plugin names (no built-in or loaded plugin has one)
8	parameters: valid unique names and ids, finite values, minValue <= defaultValue <= maxValue
9	the file's plugins are registered all or none

Versioning rules

Change	Version	Effect on existing units
Append a service to UnitHostApi or a field to UnitProcessHost	stays 2	none; new units check UNIT_HOST_HAS before using it
Append a function to UnitPlugin or UnitFactory	stays 2	none; the firmware checks the size before calling it
Change, reorder or remove any field; change a calling convention or a meaning	bump UNIT_ABI_VERSION	refused by the loader and the web UI until rebuilt
Change the firmware's plugin structures (plugins.h)	n/a	the firmware stops compiling (static asserts) until the ABI is updated

Limits

Resource	Limit
Unit code and static data	2 MB pool for all units; 256 KB per .unt file
Heap (allocate)	64 MB shared by all units (0x8C000000 to 0x90000000), never freed
Units / plugins	16 unit files; 32 plugins and 1024 parameters including the built-in ones
MIDI out	16 events per audio block, all units together
Files	read only, and only from unit_entry / createInstance
Replacing a unit	saved immediately; the new version loads at the next reboot

A minimal unit

```
#include <unit_abi.h>

#define GAIN 1                                          // parameter id
static const UnitParamInfo gainInfo = { GAIN, UNIT_PARAM_IS_AUTOMATABLE, "gain", "", 0, 2, 1 };

static float gains[4];
static int used;

static void * createInstance(void) { if (used == 4) return 0; gains[used] = 1.0F; return &gains[used++]; }
static uint32_t numIn(void) { return 1; }
static uint32_t numOut(void) { return 1; }
static uint32_t paramCount(void) { return 1; }
static int paramInfo(uint32_t i, UnitParamInfo * info) { if (i) return 0; *info = gainInfo; return 1; }
static int paramValue(void * inst, uint32_t id, double * v) {
    if (id != GAIN) return 0;
    *v = *(float *)inst;
    return 1;
}

static void process(void * inst, UnitProcessHost * host, size_t nIn, float ** in,
                    size_t nOut, float ** out, size_t frames,
                    const UnitParamEvent * events, size_t nEvents) {
    float * gain = inst;
    for (size_t e = 0; e < nEvents; e++) if (events[e].id == GAIN) *gain = events[e].value;  // 0..2
    for (size_t i = 0; i < frames; i++) out[0][i] = in[0][i] * *gain;
}

static const UnitPlugin plugin = { .size = sizeof(UnitPlugin), .name = "Gain",   // used in [units]
    .createInstance = createInstance, .getNumInputs = numIn, .getNumOutputs = numOut,
    .getParamCount = paramCount, .getParamInfo = paramInfo, .getParamValue = paramValue,
    .process = process };

const UnitPlugin * gain_plugin(const UnitHostApi * api) {   // listed in PLUGINS
    api->log("Gain ready");
    return &plugin;
}
```

Build with a short Makefile (`NAME := gain`, `BUNDLE := Gain`, `PLUGINS := gain_plugin`, `SRCS := gain.c`, `include ../unit.mk`) to get `build/arm32/gain.unt` and `build/aarch64/gain.unt`; unit.mk generates the UNIT_BUNDLE line, and `make test` runs the plugin on the Mac. Then use it in config.txt (cc0 0..127 sweeps the gain 0..2):

```
[units]
amp Gain
[audioroutes]
.Scarlett_2i2_USB:1 amp:1
amp:1 .device:1
[midimap]
.nanoKONTROL2:cc0 amp:gain
```

Units in the repository: **vumeter** (level meters on a Launchpad Mini MK3, MIDI out), **drums** (sampler that loads WAV files from /samples/drums with readDirectory/readFile/allocate), **yafr** (reverb), **clipgrid** (clip launcher), and **stock**: all four in one bundle.

Upload protocol

The web UI talks to the device console over the USB CDC serial port (or the debug UART). Machine-readable replies start with @; the console prints "> " after each command, which the UI uses as the acknowledgement for every line.

Command	Reply
`unitls`	`@unitabi <version>, then @unit <name> <bytes> <loaded|reboot|failed|new> <plugin,...> per file in /units`
`unitput <bytes>`	`@unitput ready; then base64 lines (768 characters, a multiple of 4) until unitend`
`unitend`	`@unitput ok <name> <loaded|unchanged|reboot> or @unitput error <reason>`
`unitrm <name>`	`@unitrm ok (the unit stays loaded until the next reboot)`
`cfgabort`	`leaves unitput (or cfgput) mode without saving`
`cfgplugins`	`@plugin, @plugindesc and an @param <plugin> <index> <id> <flags> <min> <max> <default> <name> [<module>] line per parameter`
`paramget <unit>`	`@paramvalue <unit> <parameter> <value> <text> per parameter with a value`
`paramset <unit>:<param> <value>`	`@paramset ok <value> <text> or @paramset error <message>`